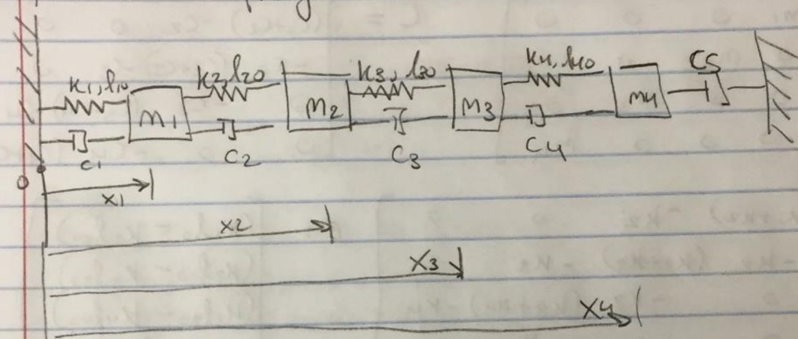
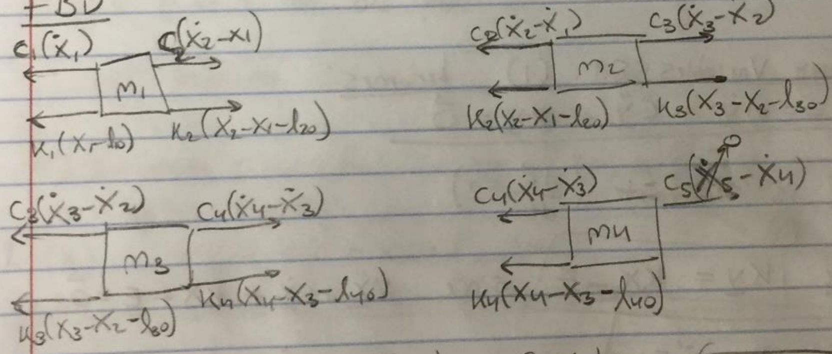
Problem 39.

a. Draw a line of n masses m_i connected to left + right walls + to eachother with various springs k_i w/ rest lengths l_{i0} + dashpots c_i. Assume no external forcing + pick which springs to leave in or out.



FBD



Equations of Motion

$$m_1\ddot{x}_1 = -(c_1+c_2)\dot{x}_1 + c_2\dot{x}_2 - (k_1+k_2)x_1 + k_2x_2 + k_1l_{10} - k_2l_{20}$$

$$m_2\ddot{x}_2 = c_2\dot{x}_1 - (c_2+c_3)\dot{x}_2 + c_3\dot{x}_3 + k_2x_1 - (k_2+k_3)x_2 + k_3x_3 + k_2l_{20} - k_3l_{30}$$

$$m_3\ddot{x}_3 = c_3\dot{x}_2 - (c_3+c_4)\dot{x}_3 + c_4\dot{x}_4 + k_3x_2 - (k_3+k_4)x_2 + k_4x_4 + k_3l_{30} - k_4l_{40}$$

$$m_4\ddot{x}_4 = c_4\dot{x}_3 - (c_4+c_5)\dot{x}_4 + k_4x_3 - k_4x_4 + k_4l_{40}$$

b. Write Eqns of Motion in Matrix form

(1) $M\ddot{\underline{X}} + C\dot{\underline{X}} + K\underline{X} = \underline{E}$

(See EOM From Part A)

$$M = \begin{bmatrix} m_1 & 0 & 0 & 0 \\ 0 & m_2 & 0 & 0 \\ 0 & 0 & m_3 & 0 \\ 0 & 0 & 0 & m_4 \end{bmatrix} \qquad C = \begin{bmatrix} (c_1+c_2) & -c_2 & 0 & 0 \\ -c_2 & (c_2+c_3) & -c_3 & 0 \\ 0 & -c_3 & (c_3+c_4) & -c_4 \\ 0 & 0 & -c_4 & (c_4+c_5) \end{bmatrix}$$

$$K = \begin{bmatrix} (k_1+k_2) & -k_2 & 0 & 0 \\ -k_2 & (k_2+k_3) & -k_3 & 0 \\ 0 & -k_3 & (k_3+k_4) & -k_4 \\ 0 & 0 & -k_4 & k_4 \end{bmatrix} \qquad E = \begin{bmatrix} (k_1 l_{10} - k_2 l_{20}) \\ (k_2 l_{20} - k_3 l_{30}) \\ (k_3 l_{30} - k_4 l_{40}) \\ k_4 l_{40} \end{bmatrix}$$

c. Change Variables s.t. (1) becomes:

(2) $== M\ddot{\underline{y}} + C\ddot{\underline{y}} + K\underline{y} = \vec{0}$.

$\rightarrow \underline{E}$ is constant $\left(\frac{d\vec{E}}{dt} = 0\right)$

$\hookrightarrow$ Try $\boxed{K\underline{y} = K\underline{X} - \underline{E}}$ since $M\ddot{\underline{X}} + C\dot{\underline{X}} + K\underline{X} - \underline{E} = \vec{0}$

$\boxed{\therefore \underline{y} = \underline{X} - K^{-1}\underline{E}}$ and to check: $\dot{\underline{y}} = \dot{\underline{X}}$, $\ddot{\underline{y}} = \ddot{\underline{X}}$ ✓

NOT YET FINALLY APPROVED

d) Pick Values for all constants and some IC w/ $\vec{X}_0$ = random constants, $\vec{V}_0 = \vec{0}$ and use ODE45 to plot position of masses as function of time.

Let $m = [1, 2, 2, 1]$

$c = [0.2, 0.3, 0.3, 0.2, 0.2]$

$k = [1, 0.5, 0.5, 1]$

$l_0 = [0.75, 0.75, 0.75, 0.75]$

$X_0 = [1, 1.75, 2.5, 3.25]$

V_0 = zeros(1, 4);

Recipe:

- Convert X_0 to $Y_0 \Rightarrow \underline{X_0} - \underline{\underline{K}} \backslash \underline{E}$;
- Plug eqn (2) $M\ddot{y} + C\dot{y} + Ky = \vec{0}$ into RHS File
- use ODE45 for tspan > 50 seconds to see full response from RHS integration
- Convert Y back to X by $\underline{X} = \underline{Y} + K \backslash \underline{E}$;
- Plot $\underline{X}$ vs. t on one plot

→ SEE MATLAB for code.

E) Introduce Sinusoidal Forcing $\vec{F}\sin(w_f t)$ to Eqn (2) to create Eqn (3):

(3) $\quad M\ddot{y} + C\dot{y} + Ky = \vec{F}\sin(w_f t)$

Use ODE45 to find the steady state response.

Let $\vec{F} = \begin{bmatrix} F_0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$; $w_f = 2$

Recipe

- using same IC's modify RHS File in part D to resemble Eqn (3).
- Integrate for t > 50 s
- Convert back to $\underline{X}$
- Plot $\underline{X}$ vs. t

SEE MATLAB for Part E. code

NOT YET FINALLY APPROVED

F) Find Homogenous and Particular Sol'n using MATLAB to evaluate an analytical Sol'n.

(4) $y = y_p + y_h$

Homogenous Sol'n

Let $\underline{z} = \begin{bmatrix} \underline{y} \\ \dot{\underline{y}} \end{bmatrix}$, $\dot{\underline{z}} = \begin{bmatrix} \dot{\underline{y}} \\ \ddot{\underline{y}} \end{bmatrix}$

$\dot{\underline{z}} = A\underline{z}$ $\quad A = \begin{bmatrix} 0^{4\times4} & I^{4\times4} \\ -M^{-1}K & -M^{-1}C \end{bmatrix}$

$\boxed{\therefore \underline{z} = \underline{z}_0 \exp(At)}$

$\rightarrow$ Convert back to $y_h \rightarrow$ $\boxed{y_h = \underline{z}(:, 1:4)}$

$\rightarrow$ See matlab for how I coded this $\leftarrow$

Particular Solution

(5) $M\ddot{\underline{y}} + C\dot{\underline{y}} + K\underline{y} = \vec{F}\sin(w_f t) \rightarrow$ Let $\underline{y_p} = \alpha\sin(w_f t) + \beta\cos(w_f t)$

$\alpha = \begin{bmatrix} A \\ B \\ C \\ D \end{bmatrix}$ $\quad \beta = \begin{bmatrix} E \\ F \\ G \\ H \end{bmatrix}$

$\alpha + \beta$ are a vectors of constants

$\therefore \dot{\underline{y}} = w_f\alpha\cos(w_f t) - w_f\beta\sin(w_f t)$

$\ddot{\underline{y}} = -w_f^2 \underline{y}$

(5) Becomes:

(6) $(K\alpha - w_f C\beta - w_f^2 M\alpha)\sin(w_f t) + (K\beta + w_f C\alpha - w_f^2 M\beta)\cos(w_f t) = \vec{F}\sin(w_f t)$

$$\boxed{\begin{aligned} &(i) \quad K\alpha - w_f C\beta - w_f^2 M\alpha = \vec{F} \\ &(ii) \quad K\beta + w_f C\alpha - w_f^2 M\beta = \vec{0} \end{aligned}}$$

$\rightarrow$ System of 8 eqns + 8 unknowns [A . . . . H]

Can Represent as $\left[N\right]^{8\times8} \begin{bmatrix} A \\ B \\ C \\ D \\ E \\ F \\ G \\ H \end{bmatrix} = \underline{d}$

$\therefore$ $\boxed{\begin{bmatrix} \alpha \\ \beta \end{bmatrix} = [N] \backslash \underline{d}}$ $\rightarrow$ $\underline{y_p}$ is now known! $\rightarrow$ SEE Symbolic code + Matlab Function for how to implement

Eqn (4) → $\underline{y}_G = \underline{y}_p + \underline{y}_h$ w/ $\underline{y}_p$ + $\underline{y}_h$ solved thru MATLAB

- Convert $\underline{y}_G$ to $\underline{X}_G$ → $\underline{X}_G = \underline{y}_G + K\backslash\underline{E}$

- Plot Response.

G) Show that the two solutions above agree → see plots

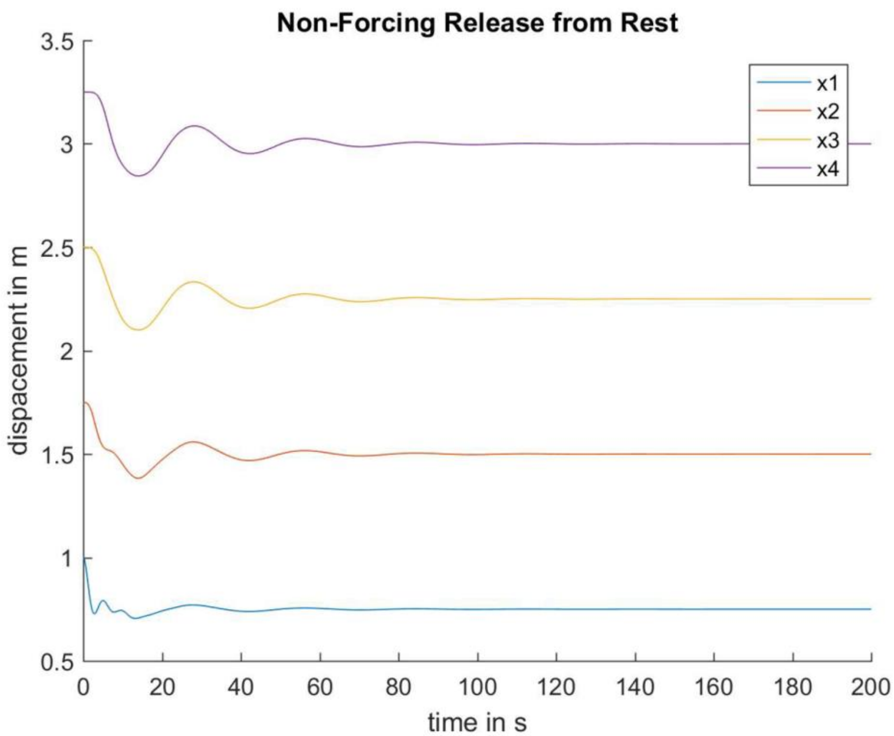
NOT YET FINALLY APPROVED

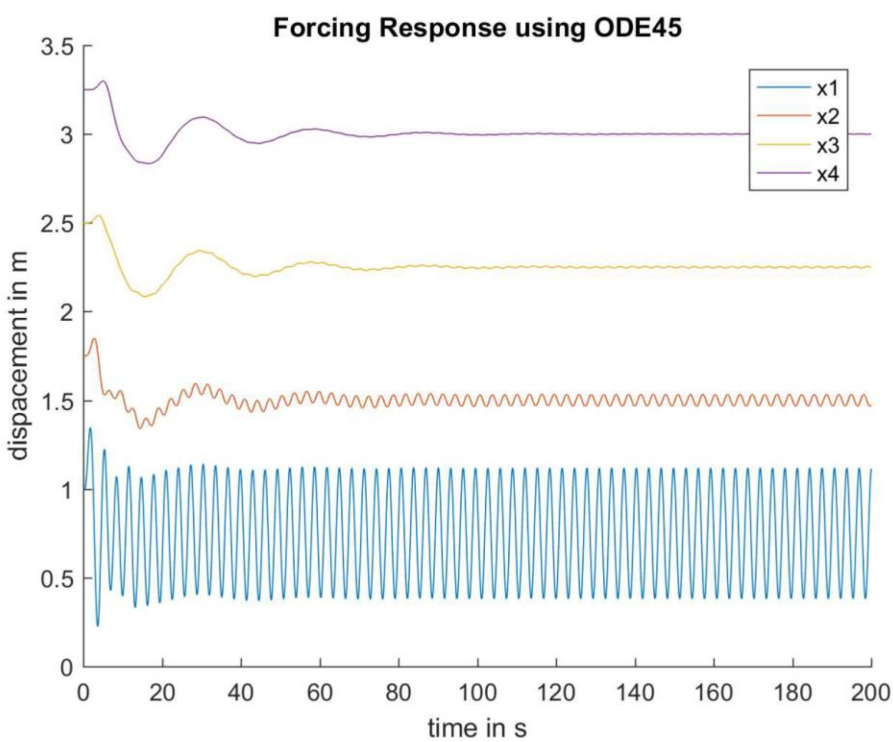
```
%problem 39
%initialize parameters
c1 = 0.2; c2 = 0.3; c3 = 0.3; c4 = c1; c5 = c1;
m1 = 1; m2 = 2; m3 = 2; m4 = 1;
k1 = 1; k2 = 0.5; k3 = 0.5; k4 = 1;
l1 = 0.75; l2 = l1; l3 = l2; l4 = l3;
%set forcing, M, C, E and K matrices
M = [m1 0 0 0; 0 m2 0 0; 0 0 m3 0; 0 0 0 m4];
C = [c1+c2 -c1 0 0; -c1 c2+c3 -c3 0; 0 -c3 c3+c4 -c4; 0 0 -c4 c4+c5];
K = [(k1+k2) -k2 0 0; -k2 (k2+k3) -k3 0; 0 -k3 (k3+k4) -k4; 0 0 -k4 k4];
E = [k1*l1-k2*l2; k2*l2-k3*l3; k3*l3-k4*l4; k4*l4];
%initial condition vectors
x0 = [1 1.75 2.5 3.25];
%change of coordinates for x0 to y0
y0 = x0'-K\E;
xd0 = [0 0 0 0]; %this is the same as yd0
IC = [y0' xd0];
options = odeset('reltol',1e-6,'abstol',1e-6);
%part D
[t,z] = ode45(@rhs_perf,[0 200],IC,options,M,C,K);
%post process
y1 = z(:,1); y2 = z(:,2); y3 = z(:,3); y4 = z(:,4);
y = [y1 y2 y3 y4];
%transform back to x coordinates
for i = 1:length(y1)
    xd(i,:) = y(i,:)+(K\E)';
end
%unpack x matrix
x1 = xd(:,1); x2 = xd(:,2); x3 = xd(:,3); x4 = xd(:,4);
%plot on the same plot
close all
hold on
plot(t,x1)
plot(t,x2)
plot(t,x3)
plot(t,x4)
legend('x1','x2','x3','x4')
xlabel('time in s')
ylabel('dispacement in m')
title('Non-Forcing Release from Rest')
%part E
F = [1 0 0 0]'; %forcing vector
w = 2; %drive frequency
[t,z] = ode45(@rhs_perf_e,[0 200],IC,options,M,C,K,F,w);
%post process
y1 = z(:,1); y2 = z(:,2); y3 = z(:,3); y4 = z(:,4);
y = [y1 y2 y3 y4];
%transform back to x coordinates
for i = 1:length(y1)
    x(i,:) = y(i,:)+(K\E)';
end
%unpack x matrix
x1 = x(:,1); x2 = x(:,2); x3 = x(:,3); x4 = x(:,4);
%plot on the same plot
figure
```

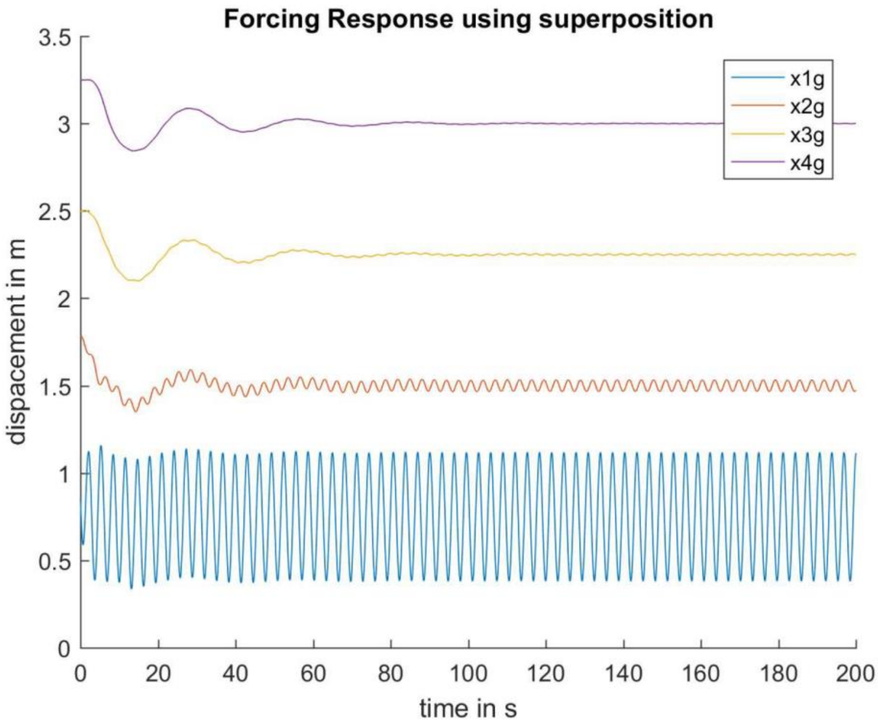
NOT YET FINALLY APPROVED

```
hold on
plot(t,x1)
plot(t,x2)
plot(t,x3)
plot(t,x4)
legend('x1','x2','x3','x4')
xlabel('time in s')
ylabel('dispacement in m')
title('Forcing Response using ODE45')
hold off
% % %Part F
A = [zeros(4,4) eye(4); -M\K -M\C];
%matlab function call (see question39e code)
[N,d] = question39e1(F,c1,c2,c3,c4,c5,k1,k2,k3,k4,m1,m2,m3,m4,w);
d = d(1:8);
coef = N\d;
for i = 1:length(t)
    %create homogeneous solution using matrix exponential
    yh(i,:) = expm(A*t(i))*(IC');
end
%create particular solution based on matlab function output
yp= coef(1:4)*sin(w*t)'+coef(5:8)*cos(w*t)';
yp = yp';
%create general solution - post process and plot
yg = yp+yh(:,1:4);
%convert back from y to x
for i = 1:length(y1)
    xg(i,:) = yg(i,:)+(K\E)';
end
%unpack x
x1g = xg(:,1); x2g = xg(:,2); x3g = xg(:,3); x4g = xg(:,4);
%plot solution
figure
hold on
plot(t,x1g)
plot(t,x2g)
plot(t,x3g)
plot(t,x4g)
xlabel('time in s')
ylabel('dispacement in m')
title('Forcing Response using superposition')
legend('x1g','x2g','x3g','x4g')
```

NOT YET FINALLY APPROVED






Forcing Response using superposition
x1g
x2g
x3g
x4g
dispacement in m
time in s
3.5
3
2.5
2
1.5
1
0.5
0
0
20
40
60
80
100
120
140
160
180
200

NOT YET FINALLY APPROVED

```
%particular solution symbolic code for part F
%Douglas Berman
syms m1 m2 m3 m4 k1 k2 k3 k4 c1 c2 c3 c4 c5 w t a b c d e f g h F co si real
%initialize the matrices
%A and B are the coefficiencts to the cos(wt) + sin(wt) particular solution
A = [a; b; c; d]; B = [e; f; g; h];
%define M C and K
M = [m1 0 0 0; 0 m2 0 0; 0 0 m3 0; 0 0 0 m4];
C = [c1+c2 -c1 0 0; -c1 c2+c3 -c3 0; 0 -c3 c3+c4 -c4; 0 0 -c4 c4+c5];
K = [(k1+k2) -k2 0 0; -k2 (k2+k3) -k3 0; 0 -k3 (k3+k4) -k4; 0 0 -k4 k4];
%F is the forcing function
F = F*sin(w*t)*[1; 0; 0; 0];
%define X, Xdot, Xddot
Y = A*sin(w*t)+B*cos(w*t);
Yd = w*A*cos(w*t)-w*B*sin(w*t);
Ydd = -w^2*Y;
%Set the general ODE into one array of equations (eqn1 = 4x1)
eqn1 = M*Ydd+C*Yd+K*Y==F;
%switch cosine and sine terms with variables co and si into new equation
eqn = subs(eqn1,[cos(w*t), sin(w*t)],[co,si]);
%Take derivative of equation wrt cosine and sine
Eqns = jacobian(eqn,[co,si]);
%equations are in terms of A and B components, and known constants (e.g. F0
%and wf)
E1 = Eqns(1:4,1);
E2 = Eqns(1:4,2);
%turn equations to matrix to be used in a backslash command
[N,y] = equationsToMatrix([E1 E2],[A' B']);
%use matlab function to create a function for ODE45 for question39 code to
%process
matlabFunction(N,y,'File','question39e1');
```

Published with MATLAB® R2016a

NOT YET FINALLY APPROVED

```
function [N,y] = question39e1(F,c1,c2,c3,c4,c5,k1,k2,k3,k4,m1,m2,m3,m4,w)
%QUESTION39E1
%    [N,Y] = QUESTION39E1(F,C1,C2,C3,C4,C5,K1,K2,K3,K4,M1,M2,M3,M4,W)

%    This function was generated by the Symbolic Math Toolbox version 7.0.
%    05-Dec-2017 16:17:37

t2 = w.^2;
t3 = k1+k2-m1.*t2;
t4 = c1+c2;
t5 = t4.*w;
t6 = k2+k3-m2.*t2;
t7 = c1.*w;
t8 = c2+c3;
t9 = t8.*w;
t10 = k3+k4-m3.*t2;
t11 = c3.*w;
t12 = c3+c4;
t13 = t12.*w;
t14 = k4-m4.*t2;
t15 = c4.*w;
t16 = c4+c5;
t17 = t16.*w;
N = reshape([t5,-c1.*w,0.0,0.0,t3,-k2,0.0,0.0,-c1.*w,t9,-c3.*w,0.0,-k2,t6,-k3,0.0,0.0,-c3.
*w,t13,-c4.*w,0.0,-k3,t10,-k4,0.0,0.0,-c4.*w,t17,0.0,0.0,-k4,t14,t3,-k2,0.0,0.0,-t5,t7,0.0
,0.0,-k2,t6,-k3,0.0,t7,-t9,t11,0.0,0.0,-k3,t10,-k4,0.0,t11,-t13,t15,0.0,0.0,-k4,t14,0.0,0.
0,t15,-t17],[8,8]);
if nargout > 1
    y = [0.0;0.0;0.0;0.0;F;0.0;0.0;0.0];
end
```

```
function zdot = rhs_perf(t,z,M,C,K)
%rhs file for part d
y1 = z(1); y2 = z(2); y3 = z(3); y4 = z(4); yd1 = z(5); yd2 = z(6); yd3 = z(7);
yd4 = z(8);
%create Y, Ydot, and Yddot vectors through equations given in scratchwork
yd = [yd1; yd2; yd3; yd4]; y = [y1; y2; y3; y4];
yDD = -M\K*y-M\C*yd;
zdot = [yd; yDD];
```

```
function zdot = rhs_perf_e(t,z,M,C,K,F,w)
%Rhs file for part E
y1 = z(1); y2 = z(2); y3 = z(3); y4 = z(4); yd1 = z(5); yd2 = z(6); yd3 = z(7);
yd4 = z(8);
yd = [yd1; yd2; yd3; yd4]; y = [y1; y2; y3; y4];
%Create Ydd based on equation 3 given in scratchwork
yDD = -M\K*y-M\C*yd+M\F*sin(w*t);
zdot = [yd; yDD];
```